

module-03-functions

Chapter 3: Functions | 41 programs in learning order

Source: Automate the Boring Stuff with Python 1e, Al Sweigart (CC-BY-NC-SA). PPT Unit-2: def, params, return, None, keyword args end/sep, local/global scope, recursion, guess-number

How to use this guide (beginner path)

1) Read Concepts once. 2) Study the Visuals - redraw each on paper. 3) Follow the Program map. 4) Work programs 01..N in order. [shell]: type every plain line into IDLE, compare with # OUT: comments (errors shown are intentional demos). [script]: save, predict output, run with F5. 5) Drill Interview prep. 6) Teach it: explain each visual aloud without looking.

Concepts in detail

A function is a mini-program: define once with `def`, call many times. Arguments are the values you pass in; parameters are the names that receive them. `return` sends a value back to the caller. A function without `return` gives back `None`. Variables created inside a function are local (invisible outside); variables outside are global (readable inside, writable only with a global statement).

Key ideas to carry forward

- `def hello(name): ... then hello('Alice')` - define, call, pass, argument, parameter.
- `len('Hello')` returns 5; your own functions return with the `return` keyword.
- `print()` returns `None` - `spam = print('Hi')` stores `None`, not `'Hi'`.
- `print('Hello', end="")` and `sep='-'` change the separator/ending (keyword arguments).
- Local scopes cannot see each other; use `return` values instead of reaching across.
- `try/except` (`ZeroDivisionError` demo) keeps one bad input from killing the program.
- `Guess-the-Number` + `Collatz` show full programs built from functions + loops.

Classic beginner mistakes

- ! Expecting a function to change a caller's variable without `return`.
- ! Using a local variable outside its function (`NameError`).
- ! Forgetting `return` so a function silently gives `None`.

Study order hint: Follow 01-08 (define/call/return/None), 10-15 (print keyword args), 16-27 (scope rules), 28-33 (errors), 34-41 (guess + collatz).

Interview prep - questions you must answer cold

Say each answer aloud in under 60 seconds, then prove it with one program from this module.

Q1. Define: function, parameter, argument, return value.

A: Function: reusable block defined with def. Parameter: receiving name in the definition. Argument: value passed at the call. Return value: what the call evaluates to via return.

Q2. What does a function return if it has no return?

A: None. So spam = print('Hi') stores None.

Q3. Explain *args/**kwargs and keyword arguments.

A: Keyword arguments are passed by name (end=", sep='-'), often optional with defaults. *args collects extra positional values, **kwargs extra named ones.

Q4. Local vs global scope?

A: Locals exist only inside their function call; globals live at module level. Locals cannot see each other; functions read globals but need a global statement to rebind them.

Q5. What is the call stack?

A: The stack of active function calls. Each call pushes a frame (its locals); return pops it. Tracebacks read top-of-stack last.

Q6. Recursion base case - why mandatory?

A: Without a base case the function calls itself forever until RecursionError. Base case (e.g. n == 0 returns 1) stops the descent in factorial.

Teach it to others

The one analogy that sticks

A function is a vending machine: arguments are coins + button pressed, parameters are the slots inside, return is the snack that drops out. Scope is hotel rooms - each guest (call) has their own towels (locals); the lobby plant (global) is shared.

What your students will get wrong

- x Calling `print()` inside a function returns the text (it returns `None`).
- x Same variable name in two functions means shared (separate locals).
- x `def` runs the code (it only defines; the call runs it).

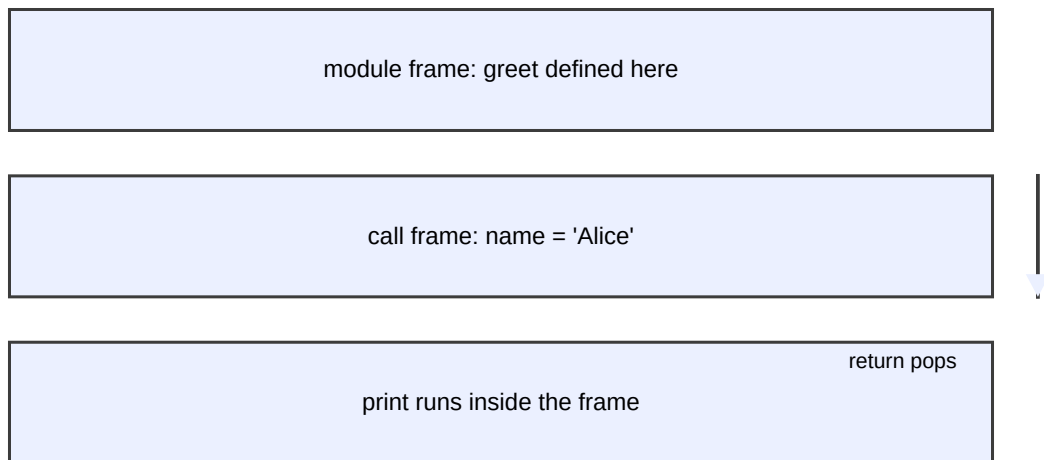
5-minute whiteboard plan

1. Draw the call stack for `hello('Alice')` frame by frame.
2. Show `print()` returning `None` live - the gasp moment.
3. Students fix a `NameError` by returning eggs instead of printing it.
4. Exit ticket: write `collatz(n)` for one step with `return`.

Visualize it - draw these from memory

Redraw each figure on paper with labels before you touch code. If you cannot draw it, you cannot teach it.

Visual - the call stack: frames push on call, pop on return



Each call gets a fresh frame (own locals). return hands one value back, then the frame is destroyed.

More examples - same idea, new clothes

Three bonus drills per module. Cover the output, predict, then run.

X1. Greeting machine

```
def greet(name, punct='!'):
    return 'Hello ' + name + punct
print(greet('Al'))
print(greet('Bo', punct='?'))
# EXPECTED: Hello Al! / Hello Bo?
```

X2. Safe divide

```
def safe_div(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return None
print(safe_div(10, 2))
print(safe_div(10, 0))
# EXPECTED: 5.0 / None
```

X3. Countdown (recursion)

```
def count(n):
    if n <= 0:
        print('Go!')
    else:
        print(n)
        count(n - 1)
count(3)
# EXPECTED: 3 2 1 Go!
```

Program map - all 41 at a glance

```
01 01_functions.py
02 02_functions_howdy.py
03 03_functions_howdy_2.py
04 04_def_statements_parameters.py
05 05_def_statements_parameters_hello.py
06 06_return_values_return.py
07 07_return_values_return_2.py
08 08_return_values_return_getanswer.py
09 09_none_value.py
10 10_keyword_arguments_print.py
11 11_keyword_arguments_print_hello.py
12 12_keyword_arguments_print_hello_2.py
13 13_keyword_arguments_print_helloworld.py
14 14_keyword_arguments_print_cats.py
15 15_keyword_arguments_print_cats_2.py
16 16_local_variables_cannot.py
17 17_local_variables_cannot_eggs.py
18 18_local_scopes_cannot.py
19 19_global_variables_can.py
20 20_local_global_variables.py
21 21_local_global_variables_bacon.py
22 22_global_statement.py
23 23_global_statement_spam.py
24 24_global_statement_spam_2.py
25 25_global_statement_spam_3.py
26 26_global_statement_spam_4.py
27 27_global_statement_spam_5.py
28 28_exception_handling.py
29 29_exception_handling_spam.py
```

30 30_exception_handling_spam_2.py
31 31_exception_handling_error.py
32 32_exception_handling_spam_3.py
33 33_exception_handling_error_2.py
34 34_short_program_guess.py
35 35_short_program_guess_random.py
36 36_short_program_guess_random_2.py
37 37_short_program_guess_i.py
38 38_short_program_guess_2.py
39 39_short_program_guess_else.py
40 40_short_program_guess_3.py
41 41_collatz_sequence.py

Topic: Functions

01/41 01_functions.py [script]

Script: save and run; predict the output first, then compare.

```
def hello():  
    print('Howdy!')  
    print('Howdy!!!')  
    print('Hello there.')  
hello()  
hello()  
hello()
```

02/41 02_functions_howdy.py [script]

Script: save and run; predict the output first, then compare.

```
Howdy!  
Howdy!!!  
Hello there.  
Howdy!  
Howdy!!!  
Hello there.  
Howdy!  
Howdy!!!  
Hello there.
```

03/41 03_functions_howdy_2.py [script]

Script: save and run; predict the output first, then compare.

```
print('Howdy!')
print('Howdy!!!')
print('Hello there.')
print('Howdy!')
print('Howdy!!!')
print('Hello there.')
print('Howdy!')
print('Howdy!!!')
print('Hello there.')
```

Topic: def Statements with Parameters

04/41 04_def_statements_parameters.py [script]

Script: save and run; predict the output first, then compare.

```
def hello(name):  
    print('Hello ' + name)  
  
hello('Alice')  
hello('Bob')
```

05/41 05_def_statements_parameters_hello.py [script]

Script: save and run; predict the output first, then compare.

```
Hello Alice  
Hello Bob
```

Topic: Return Values and return Statements

06/41 06_return_values_return.py [script]

Script: save and run; predict the output first, then compare.

```
import random
def getAnswer(answerNumber):
    if answerNumber == 1:
        return 'It is certain'
    elif answerNumber == 2:
        return 'It is decidedly so'
    elif answerNumber == 3:
        return 'Yes'
    elif answerNumber == 4:
        return 'Reply hazy try again'
    elif answerNumber == 5:
        return 'Ask again later'
    elif answerNumber == 6:
        return 'Concentrate and ask again'
    elif answerNumber == 7:
        return 'My reply is no'
    elif answerNumber == 8:
        return 'Outlook not so good'
    elif answerNumber == 9:
        return 'Very doubtful'

r = random.randint(1, 9)
fortune = getAnswer(r)
print(fortune)
```

07/41 07_return_values_return_2.py [script]

Script: save and run; predict the output first, then compare.

```
r = random.randint(1, 9)
fortune = getAnswer(r)
print(fortune)
```

08/41 08_return_values_return_getanswer.py [script]

Script: save and run; predict the output first, then compare.

```
print(getAnswer(random.randint(1, 9)))
```

Topic: The None Value

09/41 09_none_value.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
spam = print('Hello!')
```

```
# OUT: Hello!
```

```
None == spam
```

```
# OUT: True
```

Topic: Keyword Arguments and print()

10/41 10_keyword_arguments_print.py [script]

Script: save and run; predict the output first, then compare.

```
print('Hello')  
print('World')
```

11/41 11_keyword_arguments_print_hello.py [script]

Script: save and run; predict the output first, then compare.

```
Hello  
World
```

12/41 12_keyword_arguments_print_hello_2.py [script]

Script: save and run; predict the output first, then compare.

```
print('Hello', end='')  
print('World')
```

13/41 13_keyword_arguments_print_helloworld.py [script]

Script: save and run; predict the output first, then compare.

```
HelloWorld
```

14/41 14_keyword_arguments_print_cats.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
print('cats', 'dogs', 'mice')
```

```
# OUT: cats dogs mice
```

15/41 15_keyword_arguments_print_cats_2.py [shell]

Interactive: type each plain line into IDLE; # OUT: lines are expected results.

Interactive session from the book. Run line-by-line in IDLE.

Lines starting with >>> are input; other lines are expected output (commented).

```
print('cats', 'dogs', 'mice', sep=',')
```

```
# OUT: cats,dogs,mice
```

Topic: Local Variables Cannot Be Used in the Global Scope

16/41 16_local_variables_cannot.py [script]

Script: save and run; predict the output first, then compare.

```
def spam():  
    eggs = 31337  
spam()  
print(eggs)
```

17/41 17_local_variables_cannot_eggs.py [script]

Script: save and run; predict the output first, then compare.

```
Traceback (most recent call last):
```

```
  File "C:/test3784.py", line 4, in <module>
```

```
    print(eggs)
```

```
NameError: name 'eggs' is not defined
```

Topic: Local Scopes Cannot Use Variables in Other Local Scopes

18/41 18_local_scopes_cannot.py [script]

Script: save and run; predict the output first, then compare.

```
def spam():  
    eggs = 99  
    bacon()  
    print(eggs)  
  
def bacon():  
    ham = 101  
    eggs = 0  
  
spam()
```

Topic: Global Variables Can Be Read from a Local Scope

19/41 19_global_variables_can.py [script]

Script: save and run; predict the output first, then compare.

```
def spam():  
    print(eggs)  
eggs = 42  
spam()  
print(eggs)
```

Topic: Local and Global Variables with the Same Name

20/41 20_local_global_variables.py [script]

Script: save and run; predict the output first, then compare.

```
def spam():
    eggs = 'spam local'
    print(eggs) # prints 'spam local'
def bacon():
    eggs = 'bacon local'
    print(eggs) # prints 'bacon local'
    spam()
    print(eggs) # prints 'bacon local'

eggs = 'global'
bacon()
print(eggs) # prints 'global'
```

21/41 21_local_global_variables_bacon.py [script]

Script: save and run; predict the output first, then compare.

```
bacon local  
spam local  
bacon local  
global
```

Topic: The global Statement

22/41 22_global_statement.py [script]

Script: save and run; predict the output first, then compare.

```
def spam():  
    global eggs  
    eggs = 'spam'  
  
eggs = 'global'  
spam()  
print(eggs)
```

23/41 23_global_statement_spam.py [script]

Script: save and run; predict the output first, then compare.

```
spam
```

24/41 24_global_statement_spam_2.py [script]

Script: save and run; predict the output first, then compare.

```
def spam():
    global eggs
    eggs = 'spam' # this is the global

def bacon():
    eggs = 'bacon' # this is a local
def ham():
    print(eggs) # this is the global

eggs = 42 # this is the global
spam()
print(eggs)
```

25/41 25_global_statement_spam_3.py [script]

Script: save and run; predict the output first, then compare.

```
spam
```

26/41 26_global_statement_spam_4.py [script]

Script: save and run; predict the output first, then compare.

```
def spam():  
    print(eggs) # ERROR!  
    eggs = 'spam local'  
  
eggs = 'global'  
spam()
```

27/41 27_global_statement_spam_5.py [script]

Script: save and run; predict the output first, then compare.

```
Traceback (most recent call last):
  File "C:/test3784.py", line 6, in <module>
    spam()
  File "C:/test3784.py", line 2, in spam
    print(eggs) # ERROR!
UnboundLocalError: local variable 'eggs' referenced before assignment
```

Topic: Exception Handling

28/41 28_exception_handling.py [script]

Script: save and run; predict the output first, then compare.

```
def spam(divideBy):  
    return 42 / divideBy  
  
print(spam(2))  
print(spam(12))  
print(spam(0))  
print(spam(1))
```

29/41 29_exception_handling_spam.py [script]

Script: save and run; predict the output first, then compare.

```
21.0
3.5
Traceback (most recent call last):
  File "C:/zeroDivide.py", line 6, in <module>
    print(spam(0))
  File "C:/zeroDivide.py", line 2, in spam
    return 42 / divideBy
ZeroDivisionError: division by zero
```

30/41 30_exception_handling_spam_2.py [script]

Script: save and run; predict the output first, then compare.

```
def spam(divideBy):
    try:
        return 42 / divideBy
    except ZeroDivisionError:
        print('Error: Invalid argument.')

print(spam(2))
print(spam(12))
print(spam(0))
print(spam(1))
```

31/41 31_exception_handling_error.py [script]

Script: save and run; predict the output first, then compare.

```
21.0
3.5
Error: Invalid argument.
None
42.0
```

32/41 32_exception_handling_spam_3.py [script]

Script: save and run; predict the output first, then compare.

```
def spam(divideBy):  
    return 42 / divideBy  
  
try:  
    print(spam(2))  
    print(spam(12))  
    print(spam(0))  
    print(spam(1))  
except ZeroDivisionError:  
    print('Error: Invalid argument.')
```

33/41 33_exception_handling_error_2.py [script]

Script: save and run; predict the output first, then compare.

```
21.0
```

```
3.5
```

```
Error: Invalid argument.
```

Topic: A Short Program: Guess the Number

34/41 34_short_program_guess.py [script]

Script: save and run; predict the output first, then compare.

```
I am thinking of a number between 1 and 20.  
Take a guess.  
10  
Your guess is too low.  
Take a guess.  
15  
Your guess is too low.  
Take a guess.  
17  
Your guess is too high.  
Take a guess.  
16  
Good job! You guessed my number in 4 guesses!
```

35/41 35_short_program_guess_random.py [script]

Script: save and run; predict the output first, then compare.

```
# This is a guess the number game.
import random
secretNumber = random.randint(1, 20)
print('I am thinking of a number between 1 and 20.')

# Ask the player to guess 6 times.
for guessesTaken in range(1, 7):
    print('Take a guess.')
    guess = int(input())

    if guess < secretNumber:
        print('Your guess is too low.')
    elif guess > secretNumber:
        print('Your guess is too high.')
    else:
        break    # This condition is the correct guess!

if guess == secretNumber:
    print('Good job! You guessed my number in ' + str(guessesTaken) + ' guesses!')
else:
    print('Nope. The number I was thinking of was ' + str(secretNumber))
```

36/41 36_short_program_guess_random_2.py [script]

Script: save and run; predict the output first, then compare.

```
# This is a guess the number game.  
import random  
secretNumber = random.randint(1, 20)
```

37/41 37_short_program_guess_i.py [script]

Script: save and run; predict the output first, then compare.

```
print('I am thinking of a number between 1 and 20.')

# Ask the player to guess 6 times.
for guessesTaken in range(1, 7):
    print('Take a guess.')
    guess = int(input())
```

38/41 38_short_program_guess_2.py [script]

Script: save and run; predict the output first, then compare.

```
if guess < secretNumber:  
    print('Your guess is too low.')
```

```
elif guess > secretNumber:  
    print('Your guess is too high.')
```

39/41 39_short_program_guess_else.py [script]

Script: save and run; predict the output first, then compare.

```
else:  
    break    # This condition is the correct guess!
```

40/41 40_short_program_guess_3.py [script]

Script: save and run; predict the output first, then compare.

```
if guess == secretNumber:  
    print('Good job! You guessed my number in ' + str(guessesTaken) + ' guesses!')  
else:  
    print('Nope. The number I was thinking of was ' + str(secretNumber))
```

Topic: The Collatz Sequence

41/41 41_collatz_sequence.py [script]

Script: save and run; predict the output first, then compare.

Enter number :

3
10
5
16
8
4
2
1